CTE 2059 - Computer Graphic

Lecture 5 – Particle System & Shadow

University of Engineering and Technology
VIETNAM NATIONAL UNIVERSITY, HANOI

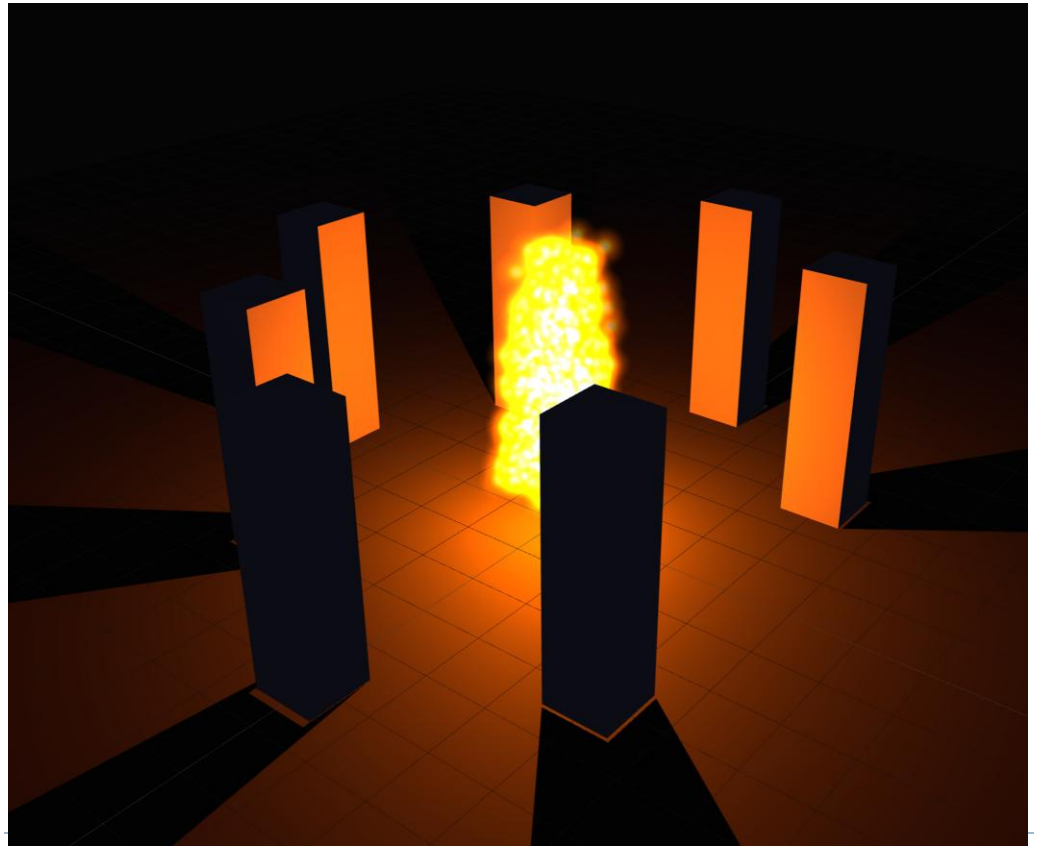
Outline

- **Getting Started**
  - Installation
- **Hệ thống hạt - Particle System**
  - Hệ thống hạt là gì?
  - Cấu trúc
  - Các bước cài đặt
- **Đổ bóng - Shadow**
  - Nguyên lý đổ bóng trong WebGL
  - Quy trình thiết lập đổ bóng
  - Nguồn sáng tạo bóng
  - Tối ưu hóa chất lượng đổ bóng

Getting Started

Installation

- Download the source code for Lecture 5 from the UET Portal
- Run the source code using Vite.



Hệ thống hạt - Particle System

Hệ thống hạt là gì?

Khái niệm

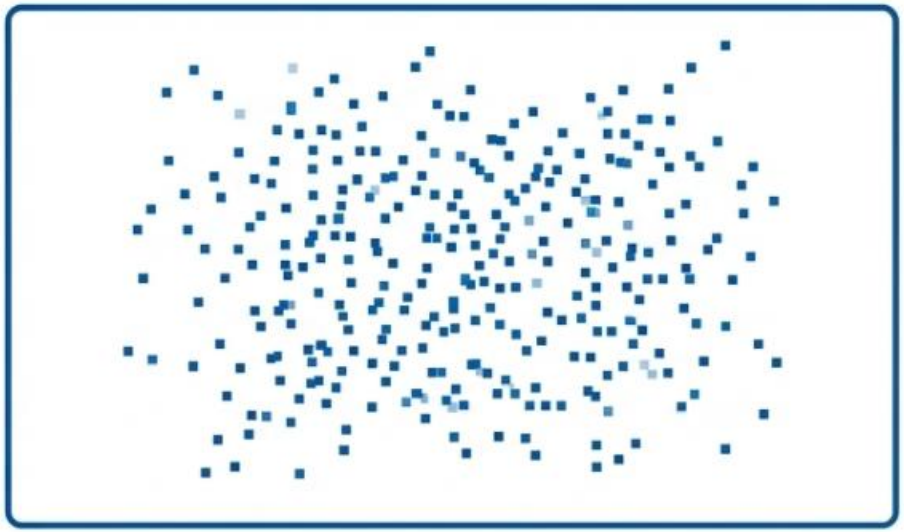
- Là kỹ thuật được sử dụng để mô phỏng các hiện tượng vật lý không có hình dáng cố định như **lửa**, **khói**, **mưa**, **tuyết**, **bụi** hoặc **dải ngân hà**.
- Hệ thống hạt cũng có thể được sử dụng để trực quan hóa dữ liệu ví dụ như **Point Clouds.**

Vấn đề hiệu suất

Nếu sử dụng 10.000 đối tượng Mesh (Box hoặc Sphere) rất nhỏ như Lec 1, CPU hoặc GPU sẽ phải thực hiện lệnh vẽ 10.000 lần.

⟹ Số lượng đa giác và **Draw Calls** quá lớn.

⟹ Trình duyệt web sẽ bị suy giảm hiệu năng.



Giải pháp: BufferGeometry & Points

THREE.Mesh

- 1 **Draw Call** cho mỗi vật thể.
- Quá tải khi số lượng vật thể quá lớn.
- Tính toán toàn bộ các mặt đa giác (**polygons**).

THREE.BufferGeometry + THREE.Points

- 1 **Draw Call** cho *n* vật thể.
- **BufferGeometry**: Lưu trữ dữ liệu về vị trí (*x, y, z*), màu sắc, và kích thước, … của từng vật thể dưới dạng các mảng dữ liệu (**attributes**).
- **Points**: **Object3D** tối ưu hóa để vẽ theo các điểm ảnh thay vì các mặt đa giác.

Cấu trúc của một hệ thống hạt

Tương tự với bài toán "Hello Cube" ở Lec 1, một hệ thống hạt cần sự gắn kết của 3 thành phần cốt lõi:

[1] Geometry
BufferGeometry (Định nghĩa vị trí trong không gian, màu sắc, và kích thước của từng hạt)

[2] Material
PointMaterial (Xác định cách các hạt hiển thị, màu sắc, độ trong suốt, texture)

[3] Object
Points (Gắn kết Geometry và Material lại thành một thực thể để thêm vào **Scene**)

Bước 1 – Khởi tạo BufferGeometry

```
// 1. Tạo dữ liệu vị trí cho 5000 hạt
const vertices = [];
for ( let i = 0; i < 5000; i ++ ) {
        const x = THREE.MathUtils.randFloatSpread( 100 );
        const y = THREE.MathUtils.randFloatSpread( 100 );
        const z = THREE.MathUtils.randFloatSpread( 100 );
        vertices.push( x, y, z );
}

const geometry = new THREE.BufferGeometry();
geometry.setAttribute( 'position', new THREE.Float32BufferAttribute( vertices, 3 ) );
```

Mỗi hạt cần 3 giá trị x, y, z liên tiếp nhau trong mảng 1 chiều.

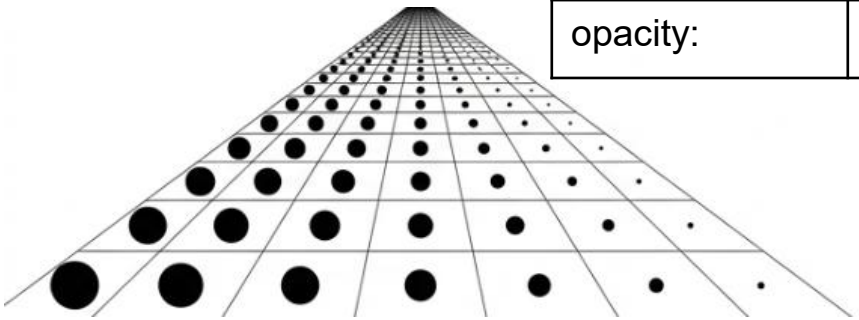
Mảng kiểu dữ liệu nguyên thủy giúp GPU có thể đọc trực tiếp từ bộ nhớ,

Bước 2 & 3 – Tạo vật liệu và Đưa vào Scene

```
// 2. Tạo Material
const material = new THREE.PointsMaterial( {
        color: 0x888888,
        size: 0.1,
        sizeAttenuation: true,
        transparent: true,
        opacity: 0.5
} );

// 3. Kết hợp thành Points
const points = new THREE.Points( geometry, material );
scene.add( points );
```

THREE.PointsMaterial

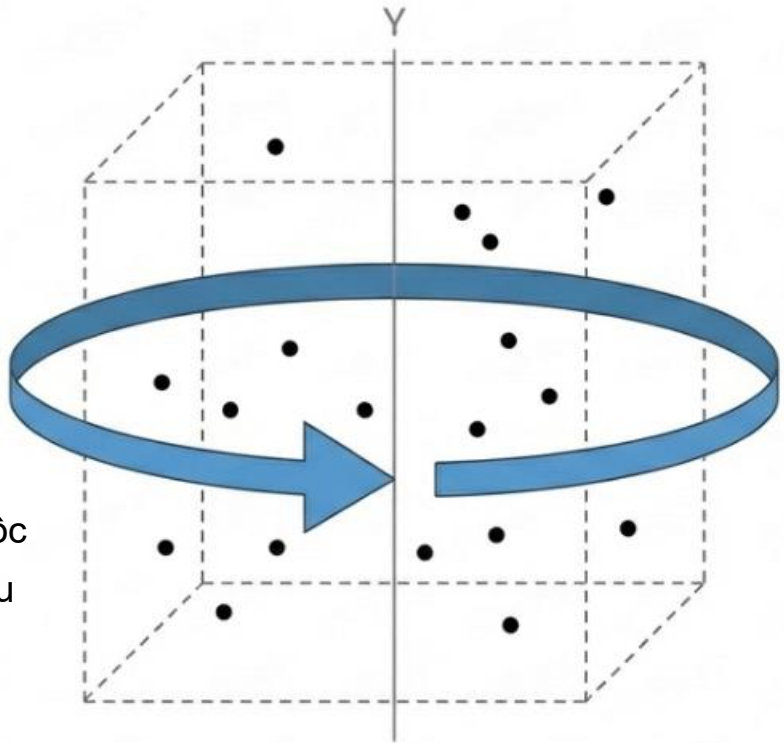
color:	Màu sắc hạt
size:	Kích thước hạt
sizeAttenuation:	(**Quan trọng**) Khi đặt bằng **True**, hạt ở xa camera sẽ nhỏ hơn, hạt gần lớn hơn, phù hợp với Perspective Camera.
transparent:	Cho phép hạt có độ trong suốt. Nếu không bật, thuộc tính **opacity** sẽ không có tác dụng
opacity:	Độ trong suốt của hạt



Chuyển động cho hạt (Animation)

Nguyên lý: Thay vì tính toán lại vị trí của từng hạt (tốn kém tài nguyên xử lý), cách đơn giản và tối ưu nhất là xoay hoặc di chuyển toàn bộ cụm hạt của hệ thống.

```
// 4. Animation
function animate() {
        requestAnimationFrame ( animate );

        points.rotation.y += 0.01;
        renderer.render( scene, camera );
}
```



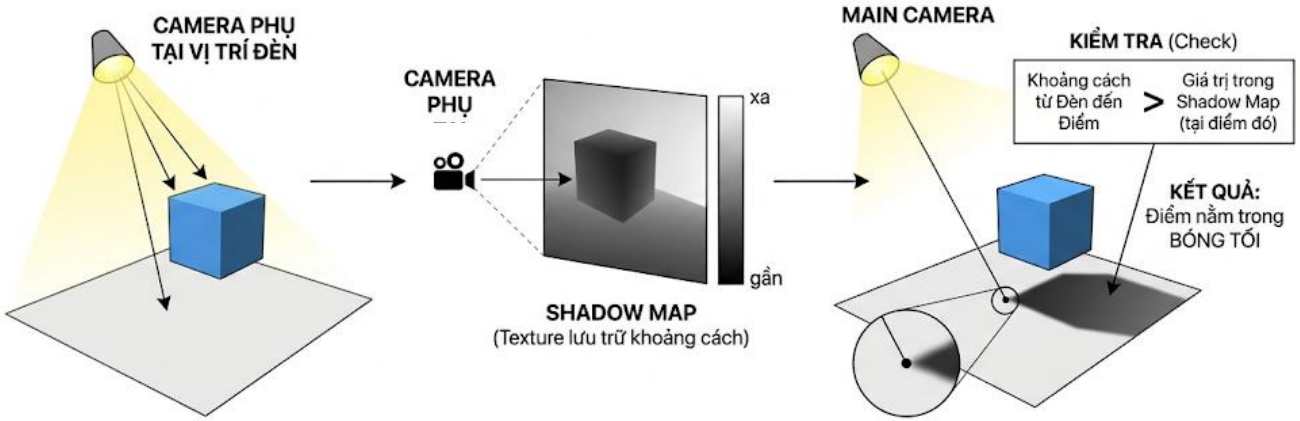
Lưu ý: Vị trí của từng hạt cũng thể được điều chỉnh độc lập bằng cách lặp qua mảng vertices để tạo ra các hiệu ứng như hạt tuyết/hạt mưa rơi độc lập.

Đổ bóng - Shadow

Nguyên lý Đổ bóng trong WebGL

Nguyên lý: Trong đồ hoạt máy tính 3D, bóng đổ không tự nhiên sinh ra. Three.js sử dụng kỹ thuật **Shadow Mapping.**

- **Cơ chế**: Một "camera" phụ được đặt tại vị trí nguồn sáng để chụp lại độ sâu (depth) của các vật thể.
- **Shadow Map:** Là một texture lưu trữ khoảng cách từ nguồn sáng đến vật thể gần nhất. Nếu một điểm trên vật thể có khoảng cách xa hơn giá trị trong Shadow Map, điểm đó nằm trong bóng tối.



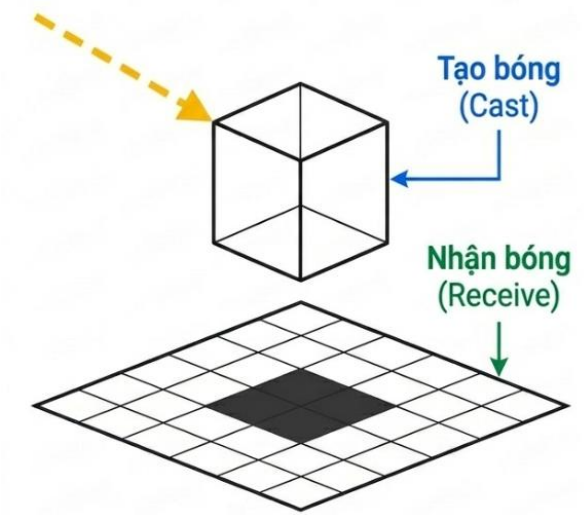
Vấn đề: Đổ bóng tiêu tốn cực kỳ nhiều tài nguyên phần cứng. Do đó, trong Three.js, tính năng này **BỊ TẮT** theo mặc định.

Quy trình thiết lập đổ bóng

Các bước thiết lập đổ bóng: Để đổ bóng hoạt động, cần kích hoạt ở 4 cấp độ

1. Renderer: Cho phép tính toán shadow map → **2. Nguồn sáng:** Một số loại đèn mới tạo được bóng → **3. Vật tạo bóng:** Các vật thể che khuất ánh sáng → **4. Vật nhận bóng:** Các vật thể bị che khuất

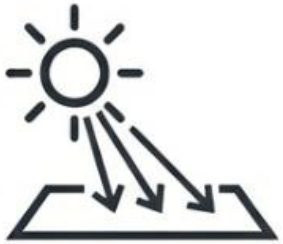
```
// Bước 1 – Kích hoạt trên WebGL Renderer
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap; // Giúp rìa bóng mịn hơn

// Bước 2 – Sử dụng nguồn sáng cho phép tạo bóng
light.castShadow = true;

// Bước 3 & 4 – Kích hoạt trên vật thể tạo và nhận bóng
mesh.castShadow = true;
ground.receiveShadow = true;
```



Nguồn sáng tạo bóng (Cast Shadow)

Quy tắc: Không phải loại nguồn sáng nào cũng có thể tạo ra bóng. Cần phải chọn đúng loại nguồn sáng có định hướng.

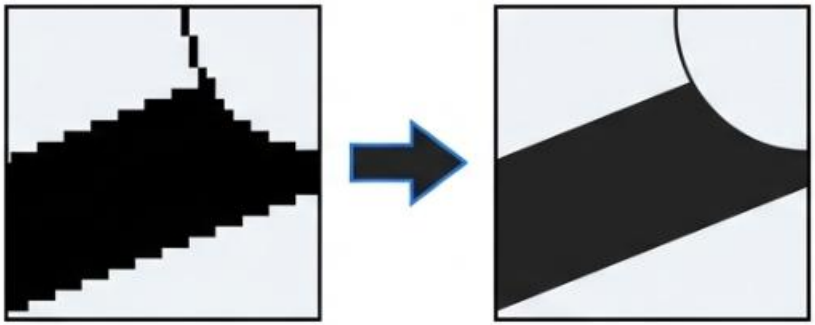
The Light Shadow Matrix	
Không Tạo Bóng	Tạo Bóng (Cast Shadow = **true**)

`AmbientLight`
`HemisphereLight`
	 `DirectionalLight`
	`PointLight`
	`SpotLight`

Ánh sáng môi trường, không có hướng chiếu cụ thể.	Đèn có hướng chiếu vật lý cụ thể.

Tối ưu hóa chất lượng đổ bóng

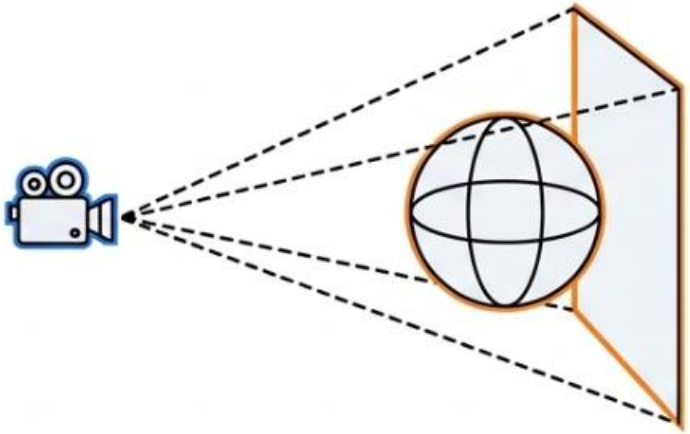
Khắc phục Bóng mờ & Răng cưa



Tăng độ phân giải của Shadow Map bằng kích thước pixel.

```
light.shadow.mapSize.width = 1024;
light.shadow.mapSize.height = 1024;
```

Chống treo máy (Giới hạn Frustum)



Giới hạn Shadow Camera. Chỉ yêu cầu máy tính tính toán bóng đổ trong phạm vi hẹp mà camera có thể nhìn thấy, loại bỏ dữ liệu thừa bên ngoài Frustum.